

SOLANA BOOTCAMP I.O

8 weeks

8 topics

~20-30 modules per week

Live interview every week

Topics

Week 1 - Rust Basics

Week 2 - Rust advanced

Week 3 - Orderbooks and CEXs

Week 4 - Defi (CLMMs/DLMMs/Perps/Tokens)

Week 5 - Client side Wallets

Week 6 - Distributed key generation

Week 7 - Indexing

Week 8 - Validator infra/internals

Week 1 - Rust

Part 1 - Rust (Easy)



- ✓ Hello Rust
 - > EASY (4)
 - > Data Types
 - > Functions & Control Flow
 - > Ownership & Borrowing
 - > Structs & Methods
 - > Enums & Pattern Matching
 - > Collections
 - > Error Handling
 - > Generics & Traits
 - > Lifetimes
- ✓ Closures & Iterators
 - > MEDIUM (3)
 - > HARD (1)


```

15
16     /// Account information
17     #[derive(Clone)]
18     #[repr(C)]
19     pub struct AccountInfo<'a> {
20         /// Public key of the account
21         pub key: &'a Pubkey,
22         /// The lamports in the account. Modifiable by programs.
23         pub lamports: Rc<RefCell<&'a mut u64>>,
24         /// The data held in this account. Modifiable by programs.
25         pub data: Rc<RefCell<&'a mut [u8]>>,
26         /// Program that owns this account
27         pub owner: &'a Pubkey,
28         /// The epoch at which this account will next owe rent
29         pub rent_epoch: Epoch,
30         /// Was the transaction signed by this account's public key?
31         pub is_signer: bool,
32         /// Is the account writable?
33         pub is_writable: bool,
34         /// This account's data contains a loaded program (and is now read-only)
35         pub executable: bool,
36     }

```

https://github.com/solana-labs/solana/blob/master/sdk/program/src/account_info.rs


```

15
16     /// Account information
17     #[derive(Clone)]
18     #[repr(C)]
19     pub struct AccountInfo<'a> {
20         /// Public key of the account
21         pub key: &'a Pubkey,
22         /// The lamports in the account. Modifiable by programs.
23         pub lamports: Rc<RefCell<&'a mut u64>>,
24         /// The data held in this account. Modifiable by programs.
25         pub data: Rc<RefCell<&'a mut [u8]>>,
26         /// Program that owns this account
27         pub owner: &'a Pubkey,
28         /// The epoch at which this account will next owe rent
29         pub rent_epoch: Epoch,
30         /// Was the transaction signed by this account's public key?
31         pub is_signer: bool,
32         /// Is the account writable?
33         pub is_writable: bool,
34         /// This account's data contains a loaded program (and is now read-only)
35         pub executable: bool,
36     }

```

https://github.com/solana-labs/solana/blob/master/sdk/program/src/account_info.rs

```

15
16     /// Account information
17     #[derive(Clone)]
18     #[repr(C)]
19     pub struct AccountInfo<'a> {
20         /// Public key of the account
21         pub key: &'a Pubkey,
22         /// The lamports in the account. Modifiable by programs.
23         pub lamports: Rc<RefCell<&'a mut u64>>,
24         /// The data held in this account. Modifiable by programs.
25         pub data: Rc<RefCell<&'a mut [u8]>>,
26         /// Program that owns this account
27         pub owner: &'a Pubkey,
28         /// The epoch at which this account will next owe rent
29         pub rent_epoch: Epoch,
30         /// Was the transaction signed by this account's public key?
31         pub is_signer: bool,
32         /// Is the account writable?
33         pub is_writable: bool,
34         /// This account's data contains a loaded program (and is now read-only)
35         pub executable: bool,
36     }

```

https://github.com/solana-labs/solana/blob/master/sdk/program/src/account_info.rs

```

15
16  /// Account information
17  #[derive(Clone)]
18  #[repr(C)]
19  ✓ pub struct AccountInfo<'a> {
20      /// Public key of the account
21      pub key: &'a Pubkey,
22      /// The lamports in the account. Modifiable by programs.
23      pub lamports: Rc<RefCell<&'a mut u64>>,
24      /// The data held in this account. Modifiable by programs.
25      pub data: Rc<RefCell<&'a mut [u8]>>,
26      /// Program that owns this account
27      pub owner: &'a Pubkey,
28      /// The epoch at which this account will next owe rent
29      pub rent_epoch: Epoch,
30      /// Was the transaction signed by this account's public key?
31      pub is_signer: bool,
32      /// Is the account writable?
33      pub is_writable: bool,
34      /// This account's data contains a loaded program (and is now read-only)
35      pub executable: bool,
36  }

```

https://github.com/solana-labs/solana/blob/master/sdk/program/src/account_info.rs


```

15
16     /// Account information
17     #[derive(Clone)]
18     #[repr(C)]
19     ✓ pub struct AccountInfo<'a> {
20         /// Public key of the account
21         pub key: &'a Pubkey,
22         /// The lamports in the account. Modifiable by programs.
23         pub lamports: Rc<RefCell<&'a mut u64>>,
24         /// The data held in this account. Modifiable by programs.
25         pub data: Rc<RefCell<&'a mut [u8]>>,
26         /// Program that owns this account
27         pub owner: &'a Pubkey,
28         /// The epoch at which this account will next owe rent
29         pub rent_epoch: Epoch,
30         /// Was the transaction signed by this account's public key?
31         pub is_signer: bool,
32         /// Is the account writable?
33         pub is_writable: bool,
34         /// This account's data contains a loaded program (and is now read-only)
35         pub executable: bool,
36     }

```

https://github.com/solana-labs/solana/blob/master/sdk/program/src/account_info.rs

```

impl Processor {
    fn _process_initialize_account(
        program_id: &Pubkey,
        accounts: &[AccountInfo],
        owner: Option<&Pubkey>,
        rent_sysvar_account: bool,
    ) -> ProgramResult {
        let account_info_iter = &mut accounts.iter();
        let new_account_info = next_account_info(account_info_iter)?;
        let mint_info = next_account_info(account_info_iter)?;
        let owner = if let Some(owner) = owner {
            owner
        } else {
            next_account_info(account_info_iter)?.key
        };
        let new_account_info_data_len = new_account_info.data_len();
        let rent = if rent_sysvar_account {
            Rent::from_account_info(next_account_info(account_info_iter))?.
        } else {
            Rent::get()?
        };

        let mut account = Account::unpack_unchecked(&new_account_info.data.borrow())?;
        if account.is_initialized() {
            return Err(TokenError::AlreadyInUse.into());
        }

        if !rent.is_exempt(new_account_info.lamports(), new_account_info_data_len) {
            return Err(TokenError::NotRentExempt.into());
        }

        let is_native_mint = Self::cmp_pubkeys(mint_info.key, &crate::native_mint::id());
        if !is_native_mint {
            Self::check_account_owner(program_id, mint_info)?;
            let _ = Mint::unpack(&mint_info.data.borrow_mut())
                .map_err(|_| Into::<ProgramError>::into(TokenError::InvalidMint))?;
        }

        account.mint = *mint_info.key;
        account.owner = *owner;
        account.close_authority = COption::None;
        account.delegate = COption::None;
    }
}

```

```
impl Processor {
```

```
fn _process_initialize_account(
    program_id: &Pubkey,
    accounts: &[AccountInfo],
    owner: Option<&Pubkey>,
    rent_sysvar_account: bool,
) -> ProgramResult {
    let account_info_iter = &mut accounts.iter();
    let new_account_info = next_account_info(account_info_iter)?;
    let mint_info = next_account_info(account_info_iter)?;
    let owner = if let Some(owner) = owner {
        owner
    } else {
        next_account_info(account_info_iter)?.key
    };
    let new_account_info_data_len = new_account_info.data_len();
    let rent = if rent_sysvar_account {
        Rent::from_account_info(next_account_info(account_info_iter))?.
    } else {
        Rent::get()?
    };

    let mut account = Account::unpack_unchecked(&new_account_info.data.borrow())?;
    if account.is_initialized() {
        return Err(TokenError::AlreadyInUse.into());
    }

    if !rent.is_exempt(new_account_info.lamports(), new_account_info_data_len) {
        return Err(TokenError::NotRentExempt.into());
    }

    let is_native_mint = Self::cmp_pubkeys(mint_info.key, &crate::native_mint::id());
    if !is_native_mint {
        Self::check_account_owner(program_id, mint_info)?;
        let _ = Mint::unpack(&mint_info.data.borrow_mut())
            .map_err(|_| Into::<ProgramError>::into(TokenError::InvalidMint))?;
    }

    account.mint = *mint_info.key;
    account.owner = *owner;
    account.close_authority = COption::None;
    account.delegate = COption::None;
```



```

impl Processor {
    fn _process_initialize_account(
        program_id: &Pubkey,
        accounts: &[AccountInfo],
        owner: Option<&Pubkey>,
        rent_sysvar_account: bool,
    ) -> ProgramResult {
        let account_info_iter = &mut accounts.iter();
        let new_account_info = next_account_info(account_info_iter)?;
        let mint_info = next_account_info(account_info_iter)?;
        let owner = if let Some(owner) = owner {
            owner
        } else {
            next_account_info(account_info_iter)?.key
        };
        let new_account_info_data_len = new_account_info.data_len();
        let rent = if rent_sysvar_account {
            Rent::from_account_info(next_account_info(account_info_iter))?.
        } else {
            Rent::get()?
        };

        let mut account = Account::unpack_unchecked(&new_account_info.data.borrow())?;
        if account.is_initialized() {
            return Err(TokenError::AlreadyInUse.into());
        }

        if !rent.is_exempt(new_account_info.lamports(), new_account_info_data_len) {
            return Err(TokenError::NotRentExempt.into());
        }

        let is_native_mint = Self::cmp_pubkeys(mint_info.key, &crate::native_mint::id());
        if !is_native_mint {
            Self::check_account_owner(program_id, mint_info)?;
            let _ = Mint::unpack(&mint_info.data.borrow_mut())
                .map_err(|_| Into::<ProgramError>::into(TokenError::InvalidMint))?;
        }

        account.mint = *mint_info.key;
        account.owner = *owner;
        account.close_authority = COption::None;
        account.delegate = COption::None;
    }
}

```

```
impl Processor {
    fn _process_initialize_account(
        program_id: &Pubkey,
        accounts: &[AccountInfo],
        owner: Option<&Pubkey>,
        rent_sysvar_account: bool,
    ) -> ProgramResult {
        let account_info_iter = &mut accounts.iter();
        let new_account_info = next_account_info(account_info_iter)?;
        let mint_info = next_account_info(account_info_iter)?;
        let owner = if let Some(owner) = owner {
            owner
        } else {
            next_account_info(account_info_iter)?.key
        };
        let new_account_info_data_len = new_account_info.data_len();
        let rent = if rent_sysvar_account {
            Rent::from_account_info(next_account_info(account_info_iter))?.
        } else {
            Rent::get()?
        };

        let mut account = Account::unpack_unchecked(&new_account_info.data.borrow())?;
        if account.is_initialized() {
            return Err(TokenError::AlreadyInUse.into());
        }

        if !rent.is_exempt(new_account_info.lamports(), new_account_info_data_len) {
            return Err(TokenError::NotRentExempt.into());
        }

        let is_native_mint = Self::cmp_pubkeys(mint_info.key, &crate::native_mint::id());
        if !is_native_mint {
            Self::check_account_owner(program_id, mint_info)?;
            let _ = Mint::unpack(&mint_info.data.borrow_mut())
                .map_err(|_| Into::<ProgramError>::into(TokenError::InvalidMint))?;
        }

        account.mint = *mint_info.key;
        account.owner = *owner;
        account.close_authority = COption::None;
        account.delegate = COption::None;
    }
}
```

```
pub type ProgramResult = ResultGeneric<(), ProgramError>;
```

Type aliases (different from associative types)

```
impl Processor {
    fn _process_initialize_account(
        program_id: &Pubkey,
        accounts: &[AccountInfo],
        owner: Option<&Pubkey>,
        rent_sysvar_account: bool,
    ) -> ProgramResult {
        let account_info_iter = &mut accounts.iter();
        let new_account_info = next_account_info(account_info_iter)?;
        let mint_info = next_account_info(account_info_iter)?;
        let owner = if let Some(owner) = owner {
            owner
        } else {
            next_account_info(account_info_iter)?.key
        };
        let new_account_info_data_len = new_account_info.data_len();
        let rent = if rent_sysvar_account {
            Rent::from_account_info(next_account_info(account_info_iter))?
        } else {
            Rent::get()?
        };

        let mut account = Account::unpack_unchecked(&new_account_info.data.borrow());
        if account.is_initialized() {
            return Err(TokenError::AlreadyInUse.into());
        }

        if !rent.is_exempt(new_account_info.lamports(), new_account_info_data_len) {
            return Err(TokenError::NotRentExempt.into());
        }

        let is_native_mint = Self::cmp_pubkeys(mint_info.key, &crate::native_mint::id());
        if !is_native_mint {
            Self::check_account_owner(program_id, mint_info)?;
            let _ = Mint::unpack(&mint_info.data.borrow_mut())
                .map_err(|_| Into::<ProgramError>::into(TokenError::InvalidMint))?;
        }

        account.mint = *mint_info.key;
        account.owner = *owner;
        account.close_authority = COption::None;
        account.delegate = COption::None;
    }
}
```

Why &mut reference?
Can I mutate the values inside the arr?
What can I mutate?


```
impl Processor {
    fn _process_initialize_account(
        program_id: &Pubkey,
        accounts: &[AccountInfo],
        owner: Option<&Pubkey>,
        rent_sysvar_account: bool,
    ) -> ProgramResult {
        let account_info_iter = &mut accounts.iter();
        let new_account_info = next_account_info(account_info_iter)?;
        let mint_info = next_account_info(account_info_iter)?;
        let owner = if let Some(owner) = owner {
            owner
        } else {
            next_account_info(account_info_iter)?.key
        };
        let new_account_info_data_len = new_account_info.data_len();
        let rent = if rent_sysvar_account {
            Rent::from_account_info(next_account_info(account_info_iter))?.
        } else {
            Rent::get()?
        };

        let mut account = Account::unpack_unchecked(&new_account_info.data.borrow());
        if account.is_initialized() {
            return Err(TokenError::AlreadyInUse.into());
        }

        if !rent.is_exempt(new_account_info.lamports(), new_account_info_data_len) {
            return Err(TokenError::NotRentExempt.into());
        }

        let is_native_mint = Self::cmp_pubkeys(mint_info.key, &crate::native_mint::id());
        if !is_native_mint {
            Self::check_account_owner(program_id, mint_info)?;
            let _ = Mint::unpack(&mint_info.data.borrow_mut())
                .map_err(|_| Into::<ProgramError>::into(TokenError::InvalidMint))?;
        }

        account.mint = *mint_info.key;
        account.owner = *owner;
        account.close_authority = COption::None;
        account.delegate = COption::None;
    }
}
```

Why &mut reference?

Can I mutate the values inside the arr?

What can I mutate?

```
/// Return the next `AccountInfo` or a `NotEnoughAccountKeys` error.
pub fn next_account_info<'a, 'b, I: Iterator<Item = &'a AccountInfo<'b>>>(
    iter: &mut I,
) -> Result<I::Item, ProgramError> {
    iter.next().ok_or(ProgramError::NotEnoughAccountKeys)
}
```

<https://github.com/rust->

[lang/rust/blob/main/library/core/src/iter/traits/iterator.rs](https://github.com/rust-lang/rust/blob/main/library/core/src/iter/traits/iterator.rs)

<https://github.com/solana-program/token/blob/main/program/src/processor.rs>


```
impl Processor {
    fn _process_initialize_account(
        program_id: &Pubkey,
        accounts: &[AccountInfo],
        owner: Option<&Pubkey>,
        rent_sysvar_account: bool,
    ) -> ProgramResult {
        let account_info_iter = &mut accounts.iter();
        let new_account_info = next_account_info(account_info_iter)?;
        let mint_info = next_account_info(account_info_iter)?;
        let owner = if let Some(owner) = owner {
            owner
        } else {
            next_account_info(account_info_iter)?.key
        };
        let new_account_info_data_len = new_account_info.data_len();
        let rent = if rent_sysvar_account {
            Rent::from_account_info(next_account_info(account_info_iter))?.
        } else {
            Rent::get()?
        };

        let mut account = Account::unpack_unchecked(&new_account_info.data.borrow());
        if account.is_initialized() {
            return Err(TokenError::AlreadyInUse.into());
        }

        if !rent.is_exempt(new_account_info.lamports(), new_account_info_data_len) {
            return Err(TokenError::NotRentExempt.into());
        }

        let is_native_mint = Self::cmp_pubkeys(mint_info.key, &crate::native_mint::id());
        if !is_native_mint {
            Self::check_account_owner(program_id, mint_info)?;
            let _ = Mint::unpack(&mint_info.data.borrow_mut())
                .map_err(|_| Into::<ProgramError>::into(TokenError::InvalidMint))?;
        }

        account.mint = *mint_info.key;
        account.owner = *owner;
        account.close_authority = COption::None;
        account.delegate = COption::None;
    }
}
```

Why &mut reference?

Can I mutate the values inside the arr?

What can I mutate?

```
/// Return the next `AccountInfo` or a `NotEnoughAccountKeys` error.
pub fn next_account_info<'a, 'b, I: Iterator<Item = &'a AccountInfo<'b>>>(
    iter: &mut I,
) -> Result<I::Item, ProgramError> {
    iter.next().ok_or(ProgramError::NotEnoughAccountKeys)
}
```

<https://github.com/rust->

[lang/rust/blob/main/library/core/src/iter/traits/iterator.rs](https://github.com/rust-lang/rust/blob/main/library/core/src/iter/traits/iterator.rs)

<https://github.com/solana-program/token/blob/main/program/src/processor.rs>

```

impl Processor {
    fn _process_initialize_account(
        program_id: &Pubkey,
        accounts: &[AccountInfo],
        owner: Option<&Pubkey>,
        rent_sysvar_account: bool,
    ) -> ProgramResult {
        let account_info_iter = &mut accounts.iter();
        let new_account_info = next_account_info(account_info_iter)?;
        let mint_info = next_account_info(account_info_iter)?;
        let owner = if let Some(owner) = owner {
            owner
        } else {
            next_account_info(account_info_iter)?.key
        };

        let new_account_info_data_len = new_account_info.data_len();
        let rent = if rent_sysvar_account {
            Rent::from_account_info(next_account_info(account_info_iter))?
        } else {
            Rent::get()?
        };

        let mut account = Account::unpack_unchecked(&new_account_info.data.borrow());
        if account.is_initialized() {
            return Err(TokenError::AlreadyInUse.into());
        }

        if !rent.is_exempt(new_account_info.lamports(), new_account_info_data_len) {
            return Err(TokenError::NotRentExempt.into());
        }

        let is_native_mint = Self::cmp_pubkeys(mint_info.key, &crate::native_mint::id());
        if !is_native_mint {
            Self::check_account_owner(program_id, mint_info)?;
            let _ = Mint::unpack(&mint_info.data.borrow_mut())
                .map_err(|_| Into::<ProgramError>::into(TokenError::InvalidMint))?;
        }

        account.mint = *mint_info.key;
        account.owner = *owner;
        account.close_authority = COption::None;
        account.delegate = COption::None;
    }
}

```

```

impl Processor {
    fn _process_initialize_account(
        program_id: &Pubkey,
        accounts: &[AccountInfo],
        owner: Option<&Pubkey>,
        rent_sysvar_account: bool,
    ) -> ProgramResult {
        let account_info_iter = &mut accounts.iter();
        let new_account_info = next_account_info(account_info_iter)?;
        let mint_info = next_account_info(account_info_iter)?;
        let owner = if let Some(owner) = owner {
            owner
        } else {
            next_account_info(account_info_iter)?.key
        };

        let new_account_info_data_len = new_account_info.data_len();
        let rent = if rent_sysvar_account {
            Rent::from_account_info(next_account_info(account_info_iter))?.
        } else {
            Rent::get()?
        };

        let mut account = Account::unpack_unchecked(&new_account_info.data.borrow());
        if account.is_initialized() {
            return Err(TokenError::AlreadyInUse.into());
        }

        if !rent.is_exempt(new_account_info.lamports(), new_account_info_data_len) {
            return Err(TokenError::NotRentExempt.into());
        }

        let is_native_mint = Self::cmp_pubkeys(mint_info.key, &crate::native_mint::id());
        if !is_native_mint {
            Self::check_account_owner(program_id, mint_info)?;
            let _ = Mint::unpack(&mint_info.data.borrow_mut())
                .map_err(|_| Into::<ProgramError>::into(TokenError::InvalidMint))?;
        }

        account.mint = *mint_info.key;
        account.owner = *owner;
        account.close_authority = COption::None;
        account.delegate = COption::None;
    }
}

```

Old approach

New approach


```

impl Processor {
    fn _process_initialize_account(
        program_id: &Pubkey,
        accounts: &[AccountInfo],
        owner: Option<&Pubkey>,
        rent_sysvar_account: bool,
    ) -> ProgramResult {
        let account_info_iter = &mut accounts.iter();
        let new_account_info = next_account_info(account_info_iter)?;
        let mint_info = next_account_info(account_info_iter)?;
        let owner = if let Some(owner) = owner {
            owner
        } else {
            next_account_info(account_info_iter)?.key
        };
        let new_account_info_data_len = new_account_info.data_len();
        let rent = if rent_sysvar_account {
            Rent::from_account_info(next_account_info(account_info_iter))?.
        } else {
            Rent::get()?
        };

        let mut account = Account::unpack_unchecked(&new_account_info.data.borrow())?;
        if account.is_initialized() {
            return Err(TokenError::AlreadyInUse.into());
        }

        if !rent.is_exempt(new_account_info.lamports(), new_account_info_data_len) {
            return Err(TokenError::NotRentExempt.into());
        }

        let is_native_mint = Self::cmp_pubkeys(mint_info.key, &crate::native_mint::id());
        if !is_native_mint {
            Self::check_account_owner(program_id, mint_info)?;
            let _ = Mint::unpack(&mint_info.data.borrow_mut())
                .map_err(|_| Into::<ProgramError>::into(TokenError::InvalidMint))?;
        }

        account.mint = *mint_info.key;
        account.owner = *owner;
        account.close_authority = COption::None;
        account.delegate = COption::None;
    }
}

```

```

impl Processor {
    fn _process_initialize_account(
        program_id: &Pubkey,
        accounts: &[AccountInfo],
        owner: Option<&Pubkey>,
        rent_sysvar_account: bool,
    ) -> ProgramResult {
        let account_info_iter = &mut accounts.iter();
        let new_account_info = next_account_info(account_info_iter)?;
        let mint_info = next_account_info(account_info_iter)?;
        let owner = if let Some(owner) = owner {
            owner
        } else {
            next_account_info(account_info_iter)?.key
        };
        let new_account_info_data_len = new_account_info.data_len();
        let rent = if rent_sysvar_account {
            Rent::from_account_info(next_account_info(account_info_iter))?
        } else {
            Rent::get()?
        };

        let mut account = Account::unpack_unchecked(new_account_info.data.borrow());
        if account.is_initialized() {
            return Err(TokenError::AlreadyInUse.into());
        }

        if !rent.is_exempt(new_account_info.lamports(), new_account_info_data_len) {
            return Err(TokenError::NotRentExempt.into());
        }

        let is_native_mint = Self::cmp_pubkeys(mint_info.key, &crate::native_mint::id());
        if !is_native_mint {
            Self::check_account_owner(program_id, mint_info)?;
            let _ = Mint::unpack(&mint_info.data.borrow_mut())
                .map_err(|_| Into::<ProgramError>::into(TokenError::InvalidMint))?;
        }

        account.mint = *mint_info.key;
        account.owner = *owner;
        account.close_authority = COption::None;
        account.delegate = COption::None;
    }
}

```

```

/// Account data.
#[repr(C)]
#[derive(Clone, Copy, Debug, Default, PartialEq)]
pub struct Account {
    /// The mint associated with this account
    pub mint: Pubkey,
    /// The owner of this account.
    pub owner: Pubkey,
    /// The amount of tokens this account holds.
    pub amount: u64,
    /// If `delegate` is `Some` then `delegated_amount` represents
    /// the amount authorized by the delegate
    pub delegate: COption<Pubkey>,
    /// The account's state
    pub state: AccountState,
    /// If `is_native.is_some`, this is a native token, and the value logs the
    /// rent-exempt reserve. An Account is required to be rent-exempt, so
    /// the value is used by the Processor to ensure that wrapped SOL
    /// accounts do not drop below this threshold.
    pub is_native: COption<u64>,
    /// The amount delegated
    pub delegated_amount: u64,
    /// Optional authority to close the account.
    pub close_authority: COption<Pubkey>,
}

impl Pack for Account {
    const LEN: usize = 165;

    fn unpack_from_slice(src: &[u8]) -> Result<Self, ProgramError> {
        let src = array_ref![src, 0, 165];
        let (mint, owner, amount, delegate, state, is_native, delegated_amount, close_authority) =
            array_refs![src, 32, 32, 8, 36, 1, 12, 8, 36];
        Ok(Account {
            mint: Pubkey::new_from_array(*mint),
            owner: Pubkey::new_from_array(*owner),
            amount: u64::from_le_bytes(*amount),
            delegate: unpack_coption_key(delegate)?,
            state: AccountState::try_from_primitive(state[0])
                .or(Err(ProgramError::InvalidAccountData))?,
            is_native: unpack_coption_u64(is_native)?,
            delegated_amount: u64::from_le_bytes(*delegated_amount),
            close_authority: unpack_coption_key(close_authority)?,
        })
    }

    /// Unpack from slice without checking if initialized
    fn unpack_unchecked(input: &[u8]) -> Result<Self, ProgramError> {
        if input.len() != Self::LEN {
            return Err(ProgramError::InvalidAccountData);
        }
        Self::unpack_from_slice(input)
    }
}

```

https://github.com/solana-labs/solana/blob/master/sdk/program/src/program_pack.rs

<https://github.com/solana-program/token/blob/main/program/src/processor.rs>


```
impl Processor {
    fn _process_initialize_account(
        program_id: &Pubkey,
        accounts: &[AccountInfo],
        owner: Option<&Pubkey>,
        rent_sysvar_account: bool,
    ) -> ProgramResult {
        let account_info_iter = &mut accounts.iter();
        let new_account_info = next_account_info(account_info_iter)?;
        let mint_info = next_account_info(account_info_iter)?;
        let owner = if let Some(owner) = owner {
            owner
        } else {
            next_account_info(account_info_iter)?.key
        };
        let new_account_info_data_len = new_account_info.data_len();
        let rent = if rent_sysvar_account {
            Rent::from_account_info(next_account_info(account_info_iter)?)?
        } else {
            Rent::get()?
        };

        let mut account = Account::unpack_unchecked(&new_account_info.data.borrow()),
        if account.is_initialized() {
            return Err(TokenError::AlreadyInUse.into());
        }

        if !rent.is_exempt(new_account_info.lamports(), new_account_info_data_len) {
            return Err(TokenError::NotRentExempt.into());
        }

        let is_native_mint = Self::cmp_pubkeys(mint_info.key, &crate::native_mint::id());
        if !is_native_mint {
            Self::check_account_owner(program_id, mint_info)?;
            let _ = Mint::unpack(&mint_info.data.borrow_mut())
                .map_err(|_| Into::<ProgramError>::into(TokenError::InvalidMint))?;
        }

        account.mint = *mint_info.key;
        account.owner = *owner;
        account.close_authority = COption::None;
        account.delegate = COption::None;
    }
}
```

```
pub const MAX_PERMITTED_DATA_INCREASE: usize = 1_024 * 10;

/// Account information
#[derive(Clone)]
#[repr(C)]
pub struct AccountInfo<'a> {
    /// Address of the account
    pub key: &'a Address,
    /// The lamports in the account. Modifiable by programs.
    pub lamports: Rc<RefCell<&'a mut u64>>,
    /// The data held in this account. Modifiable by programs.
    pub data: Rc<RefCell<&'a mut [u8]>>,
    /// Program that owns this account
    pub owner: &'a Address,
    /// Formerly, the epoch at which this account will next owe rent. A field
    /// must remain because the runtime depends on the exact layout of this
    /// struct.
    #[deprecated(
        since = "3.0.0",
        note = "Do not use this field, it will not exist in ABIv2"
    )]
    pub _unused: u64,
    /// Was the transaction signed by this account's public key?
    pub is_signer: bool,
    /// Is the account writable?
    pub is_writable: bool,
    /// This account's data contains a loaded program (and is now read-only)
    pub executable: bool,
}
```